

用户视角下的龙门逻辑5.9

问题描述

站在用户角度，看下用户到底需要什么样的龙门控制逻辑。

这样就确定了龙门轴的控制逻辑。然后再自顶向下，逐步确认逻辑即可。

这里描述的同时，不仅有用户视角下的龙门逻辑，还包括了基于 PLC 寄存器的控制和状态逻辑。

一、默认的联动状态

用户视角

用户切换到同步龙门轴的时候，要使用的就是逻辑轴X，即联动状态下的X1X2。

并且解除联动作为相当危险的操作：

- 单独移动X1、X2：导致X1和X2之间连接的物理横杆歪曲

所以在解除联动的时候，需要输入管理员密码。

这里 UI 设计我思考的是：

- 用户切换到龙门轴X1X2界面
 - 界面显示状态：X1X2是否使能、是否联动、位置信息、速度信息等
 - 界面展示一个名为《同步联动》的绿色按钮和一个《解除同步》的红色按钮
 - 除此之外界面置灰，除《同步联动》、《解除同步》两个按钮外界面均不可控制。
- 点击《同步联动》按钮
 - 此时后台执行使能+联动的逻辑
 - 如果X1X2满足联动执行条件且PLC确实处于了联动状态，则界面显示状态X1X2均使能且处于联动状态
 - 此时界面解除联动状态的控制按钮：
 - X点动
 - X位置移动等
 - 此时可以进行X逻辑轴（X1X2联动）的运动逻辑。
- 点击《解除同步》按钮
 - 此时警告用户超差风险，并且强制用户输入两次管理员密码
 - 输入成功后，展示X1和X2单独控制逻辑界面，可以分别进行控制。

当然这里是我自己的思考，还得需要根据实际情况再做调整。

PLC 视角

要想让两个电机联动，就必须 PLC 先后执行两个控制寄存器的操作：

- 使能轴X电机 ON （此操作会让X1、X2电机同时使能）
- 轴X联动使能 ON

PLC 在寄存器《轴X联动使能》写入 ON 后，会自动检测X1、X2位置超差是否满足联动使能条件。

如果满足联动条件，则会输出 PLC 状态寄存器：

- 轴X联动状态 ON

否则

- 轴X联动状态 OFF

PLC 的状态寄存器 《轴X联动状态》梯形逻辑如下：

```
1 // 联动开启
2 IF 轴X1CurSpeed = 0.0 AND 轴X2CurSpeed = 0.0 AND 轴X1Status AND 轴X2Status THEN
3     IF 轴X联动使能 AND ABS(轴X1CurPosition + 轴X2CurPosition) < 0.1 THEN
4         X轴关联触发 := 轴X联动使能;
5     END_IF;
6 END_IF;
7 // 联动关闭
8 IF NOT 轴X联动使能 THEN
9     X轴关联触发 := 轴X联动使能;
10 END_IF;
11
12 X轴解除关联触发 := NOT 轴X联动使能;
13
14 轴X联动状态 := X轴关联触发;
```

二、联动状态下的逻辑

这里首先整理下联动状态下 PLC 的控制逻辑，等总结完毕后，再分析用户状态下的逻辑处理。

运动控制：

- X点动正转
- X点动反转
- 轴X绝对定位触发
- 轴X相对定位触发

状态展示（包括PLC的控制逻辑，需要AI协助确定每一个X状态的具体属性，因为现在丢失了状态寄存器的逻辑）：

- X轴急停触发

```
1 // 轴X 绝对 与 相对 定位 接口 必须在X1 X2联动的情况下才能执行
2 轴X暂停沿触发(CLK := 轴X软件限位状态,Q => 轴X暂停触发结果);
3 IF 轴X软件限位状态 THEN
4     轴X1MoveDo := FALSE;
5     轴X1MoveXDDo := false;
```

```

6  END_IF;
7
8  if 齿轮比到达 AND
9      轴X联动使能 AND NOT 解除关联完成 AND 轴X1CurSpeed = 0.0 AND
10     NOT 轴X1MoveDo AND NOT 轴X1MoveXDDo and not 轴X1HomeDo and NOT 轴X1ZeroDo and
11     NOT 轴X1JogForward AND NOT 轴X1JogBackward AND NOT X轴急停触发 THEN
12
13     IF not 轴X软件限位状态 and
14         轴X绝对定位距离 <> 轴X1CurPosition AND 轴X绝对定位触发 AND NOT 轴X相对定位触发 then
15         轴X1MovePosition := 轴X绝对定位距离;
16         轴X1MoveDo := TRUE;
17     END_IF;
18     轴X相对定位沿触发(CLK := 轴X相对定位触发,Q => 轴X相对定位沿触发结果);
19     IF not 轴X软件限位状态 and
20         轴X相对定位距离 <> 0.0 AND not 轴X绝对定位触发 and 轴X相对定位沿触发结果 then
21         轴X1MoveXDPosition := 轴X相对定位距离;
22         轴X1MoveXDDo := TRUE;
23     END_IF;
24
25 END_IF;

```

```

1  // 轴X 点动 接口
2  IF 轴X软件限位状态 THEN
3      IF 轴X1CurPosition <= 轴X软件负限位 THEN 轴X1点动反转 := FALSE; END_IF;
4      IF 轴X1CurPosition >= 轴X软件正限位 THEN 轴X1点动正转 := FALSE; end_if ;
5  END_IF;
6
7  if ((齿轮比到达 AND
8      轴X联动使能 AND NOT 解除关联完成)OR NOT 轴X联动使能) AND
9      NOT 轴X1MoveDo AND NOT 轴X1MoveXDDo and not 轴X1HomeDo and NOT 轴X1ZeroDo AND 轴X1Status
10     AND NOT X轴超差时间到 AND NOT X轴急停触发 THEN
11     轴X1JogBackward := 轴X1点动反转;
12     轴X1JogForward := 轴X1点动正转;
13 END_IF;
14
15 IF NOT 齿轮比到达 AND NOT 轴X联动使能 AND NOT X轴超差时间到 AND
16     NOT 轴X2MoveDo AND NOT 轴X2MoveXDDo and not 轴X2HomeDo and NOT 轴X2ZeroDo AND 轴X2Status THEN
17     轴X2JogBackward := 轴X2点动反转;
18     轴X2JogForward := 轴X2点动正转;
19 END_IF;

```

```

1  // 联合定位的时候，两轴的位置超差就需要紧急停机
2  if 齿轮比到达 AND 轴X联动使能 AND NOT 解除关联完成 THEN
3      if ABS(轴X1CurPosition + 轴X2CurPosition) > 轴X超差阈值 THEN
4          X轴超差标志 := TRUE;
5      ELSE
6          X轴超差标志 := FALSE;
7      END_IF;
8  ELSE
9      X轴超差标志 := FALSE;
10     轴X1E_Stop := FALSE;
11 end_if;
12
13 TONR(IN := X轴超差标志,PT := 20,R := ,Q => X轴超差时间到,ET => );

```

```

14  if X轴超差时间到 THEN
15      X轴急停触发 := TRUE;
16  END_IF;
17  IF X轴急停触发 THEN
18      轴X绝对定位触发 := FALSE;
19      轴X相对定位触发 := FALSE;
20
21      轴X1点动反转 := FALSE ;
22      轴X1点动正转 := FALSE ;
23
24      轴X1MoveDo := FALSE;
25      轴X1MoveXDDo := FALSE;
26      轴X1HomeDo := FALSE;
27      轴X1ZeroDo := FALSE;
28  END_IF;
29
30
31  急停触发沿(CLK := X轴急停触发 OR 轴X软件限位状态,Q => 轴X1E_Stop);

```

■ X轴超差标志

```

1  // 联合定位的时候，两轴的位置超差就需要紧急停机
2  if 齿轮比到达 AND 轴X联动使能 AND NOT 解除关联完成 THEN
3      if ABS(轴X1CurPosition + 轴X2CurPosition) > 轴X超差阈值 THEN
4          X轴超差标志 := TRUE;
5      ELSE
6          X轴超差标志 := FALSE;
7      END_IF;
8  ELSE
9      X轴超差标志 := FALSE;
10     轴X1E_Stop := FALSE;
11 end_if;
12
13 TONR(IN := X轴超差标志,PT := 20,R := ,Q => X轴超差时间到,ET => );
14 if X轴超差时间到 THEN
15     X轴急停触发 := TRUE;
16 END_IF;
17 IF X轴急停触发 THEN
18     轴X绝对定位触发 := FALSE;
19     轴X相对定位触发 := FALSE;
20
21     轴X1点动反转 := FALSE ;
22     轴X1点动正转 := FALSE ;
23
24     轴X1MoveDo := FALSE;
25     轴X1MoveXDDo := FALSE;
26     轴X1HomeDo := FALSE;
27     轴X1ZeroDo := FALSE;
28 END_IF;
29
30
31 急停触发沿(CLK := X轴急停触发 OR 轴X软件限位状态,Q => 轴X1E_Stop);

```

■ X轴超差时间到

```

1 // 轴X 点动 接口
2 IF 轴X软件限位状态 THEN
3     IF 轴X1CurPosition <= 轴X软件负限位 THEN 轴X1点动反转 := FALSE; END_IF;
4     IF 轴X1CurPosition >= 轴X软件正限位 THEN 轴X1点动正转 := FALSE; end_if ;
5 END_IF;
6
7 if ((齿轮比到达 AND
8     轴X联动使能 AND NOT 解除关联完成)OR NOT 轴X联动使能) AND
9     NOT 轴X1MoveDo AND NOT 轴X1MoveXDDo and not 轴X1HomeDo and NOT 轴X1ZeroDo AND 轴X1Status
10    AND NOT X轴超差时间到 AND NOT X轴急停触发 THEN
11    轴X1JogBackward := 轴X1点动反转;
12    轴X1JogForward := 轴X1点动正转;
13 END_IF;
14
15 IF NOT 齿轮比到达 AND NOT 轴X联动使能 AND NOT X轴超差时间到 AND
16    NOT 轴X2MoveDo AND NOT 轴X2MoveXDDo and not 轴X2HomeDo and NOT 轴X2ZeroDo AND 轴X2Status THEN
17    轴X2JogBackward := 轴X2点动反转;
18    轴X2JogForward := 轴X2点动正转;
19 END_IF;

```

■ 轴X状态显示

```

1 // 状态显示 0: 未使能 - 停止 1: 使能（未运动） - 空闲 2: 使能在运动 - 正在运动 3: 报警（报警解除后回到1空
  闲）- 超限位（点动解除）
2 IF NOT 轴X1Status OR NOT 轴X2Status THEN
3     轴X状态显示 := 0;
4 ELSE
5     IF 轴X1CurSpeed <> 0.0 OR 轴X2CurSpeed <> 0.0 THEN
6         轴X状态显示 := 2;
7     ELSE
8         轴X状态显示 := 1;
9     END_IF;
10 END_IF;
11 IF 轴X1指令错误 OR 轴X1伺服错误码<>0 or 轴X1错误码<>0 or 轴X1伺服DIStatus = 1 or 轴X1伺服DIStatus = 2
  or
12    轴X2指令错误 OR 轴X2伺服错误码<>0 or 轴X2错误码<>0 or 轴X2伺服DIStatus = 1 or 轴X2伺服DIStatus =
  2
13    OR X轴超差时间到 then
14    轴X状态显示 := 3;
15 end_if;
16 if Axis_X1.bnslimit OR Axis_X1.bpslimit
17    OR Axis_X2.bnslimit OR Axis_X2.bpslimit THEN
18    轴X状态显示 := 3;
19 END_IF;

```

```

1 IF 轴X状态显示 = 3 THEN
2     IF 轴X1指令错误 OR 轴X2指令错误 THEN 轴X报警代码 :=1; END_IF; // 指令执行报警
3     IF 轴X1伺服错误码<>0 OR 轴X2伺服错误码<>0 THEN 轴X报警代码 :=2; END_IF; // 伺服报警
4     IF 轴X1伺服DIStatus<>0 OR 轴X2伺服DIStatus<>0
5         Axis_X1.bnslimit OR Axis_X1.bpslimit OR
6         Axis_X2.bnslimit OR Axis_X2.bpslimit THEN
7         轴X报警代码 :=3;
8         轴X联动使能 := FALSE;
9     END_IF; // 限位报警
10    IF X轴超差时间到 THEN 轴X报警代码 :=4; END_IF; // 超差报警

```

```

11
12 ELSE
13     轴X报警代码:=0;
14 END_IF;

```

■ 轴X运动完成

```

1 IF 轴X1Status AND 轴X2Status THEN
2     IF 轴X1MoveDo OR 轴X1MoveXDDo OR 轴X1JogBackward OR 轴X1JogForward OR
3         轴X2MoveDo OR 轴X2MoveXDDo OR 轴X2JogBackward OR 轴X2JogForward THEN
4         轴X运动完成 := false;
5     else
6         轴X运动完成 := true;
7     END_IF;
8     轴X绝对定位中 := 轴X1MoveDo OR 轴X2MoveDo;
9     轴X相对定位中 := 轴X1MoveXDDo OR 轴X2MoveXDDo;
10    轴X点动中 := 轴X1JogBackward OR 轴X1JogForward OR 轴X2JogBackward OR 轴X2JogForward;
11 ELSE
12    轴X绝对定位中 := FALSE;
13    轴X相对定位中 := FALSE;
14    轴X点动中 := FALSE;
15    轴X运动完成 := false;
16 END_IF;
17 if Axis_X1.bpslimit OR Axis_X1.bnslimit OR Axis_X1.bphlimit OR Axis_X1.bnhlimit THEN
18     轴X1MoveDo:= FALSE;
19     轴X1MoveXDDo:=FALSE;
20 END_IF;
21 if Axis_X1.bpslimit OR Axis_X1.bphlimit then 轴X1点动正转:=false; end_if;
22 if Axis_X1.bnslimit OR Axis_X1.bnhlimit THEN 轴X1点动反转:=FALSE; END_IF;
23 IF Axis_X2.bpslimit OR Axis_X2.bphlimit then 轴X2点动正转:=false; end_if;
24 if Axis_X2.bnslimit OR Axis_X2.bnhlimit THEN 轴X2点动反转:=FALSE; END_IF;

```

■ 轴X报警代码

```

1 IF 轴X状态显示 = 3 THEN
2     IF 轴X1指令错误 OR 轴X2指令错误 THEN 轴X报警代码 :=1; END_IF; // 指令执行报警
3     IF 轴X1伺服错误码<>0 OR 轴X2伺服错误码<>0 THEN 轴X报警代码 :=2; END_IF; // 伺服报警
4     IF 轴X1伺服DIStatus<>0 OR 轴X2伺服DIStatus<>0 OR
5         Axis_X1.bnslimit OR Axis_X1.bpslimit OR
6         Axis_X2.bnslimit OR Axis_X2.bpslimit THEN
7         轴X报警代码 :=3;
8         轴X联动使能 := FALSE;
9     END_IF; // 限位报警
10    IF X轴超差时间到 THEN 轴X报警代码 :=4; END_IF; // 超差报警
11
12 ELSE
13     轴X报警代码:=0;
14 END_IF;

```

■ 轴X绝对定位中

```

1 IF 轴X1Status AND 轴X2Status THEN
2     IF 轴X1MoveDo OR 轴X1MoveXDDo OR 轴X1JogBackward OR 轴X1JogForward OR
3         轴X2MoveDo OR 轴X2MoveXDDo OR 轴X2JogBackward OR 轴X2JogForward THEN
4         轴X运动完成 := false;

```

```

5      else
6          轴X运动完成 := true;
7      END_IF;
8      轴X绝对定位中 := 轴X1MoveDo OR 轴X2MoveDo;
9      轴X相对定位中 := 轴X1MoveXDDo OR 轴X2MoveXDDo;
10     轴X点动中 := 轴X1JogBackward OR 轴X1JogForward OR 轴X2JogBackward OR 轴X2JogForward;
11 ELSE
12     轴X绝对定位中 := FALSE;
13     轴X相对定位中 := FALSE;
14     轴X点动中 := FALSE;
15     轴X运动完成 := false;
16 END_IF;

```

■ 轴X相对定位中

```

1  IF 轴X1Status AND 轴X2Status THEN
2      IF 轴X1MoveDo OR 轴X1MoveXDDo OR 轴X1JogBackward OR 轴X1JogForward OR
3          轴X2MoveDo OR 轴X2MoveXDDo OR 轴X2JogBackward OR 轴X2JogForward THEN
4          轴X运动完成 := false;
5      else
6          轴X运动完成 := true;
7      END_IF;
8      轴X绝对定位中 := 轴X1MoveDo OR 轴X2MoveDo;
9      轴X相对定位中 := 轴X1MoveXDDo OR 轴X2MoveXDDo;
10     轴X点动中 := 轴X1JogBackward OR 轴X1JogForward OR 轴X2JogBackward OR 轴X2JogForward;
11 ELSE
12     轴X绝对定位中 := FALSE;
13     轴X相对定位中 := FALSE;
14     轴X点动中 := FALSE;
15     轴X运动完成 := false;
16 END_IF;

```

■ 轴X点动中

```

1  IF 轴X1Status AND 轴X2Status THEN
2      IF 轴X1MoveDo OR 轴X1MoveXDDo OR 轴X1JogBackward OR 轴X1JogForward OR
3          轴X2MoveDo OR 轴X2MoveXDDo OR 轴X2JogBackward OR 轴X2JogForward THEN
4          轴X运动完成 := false;
5      else
6          轴X运动完成 := true;
7      END_IF;
8      轴X绝对定位中 := 轴X1MoveDo OR 轴X2MoveDo;
9      轴X相对定位中 := 轴X1MoveXDDo OR 轴X2MoveXDDo;
10     轴X点动中 := 轴X1JogBackward OR 轴X1JogForward OR 轴X2JogBackward OR 轴X2JogForward;
11 ELSE
12     轴X绝对定位中 := FALSE;
13     轴X相对定位中 := FALSE;
14     轴X点动中 := FALSE;
15     轴X运动完成 := false;
16 END_IF;

```

■ 轴X当前绝对位置

```

1  轴X当前绝对位置:=轴X1CurPosition;

```

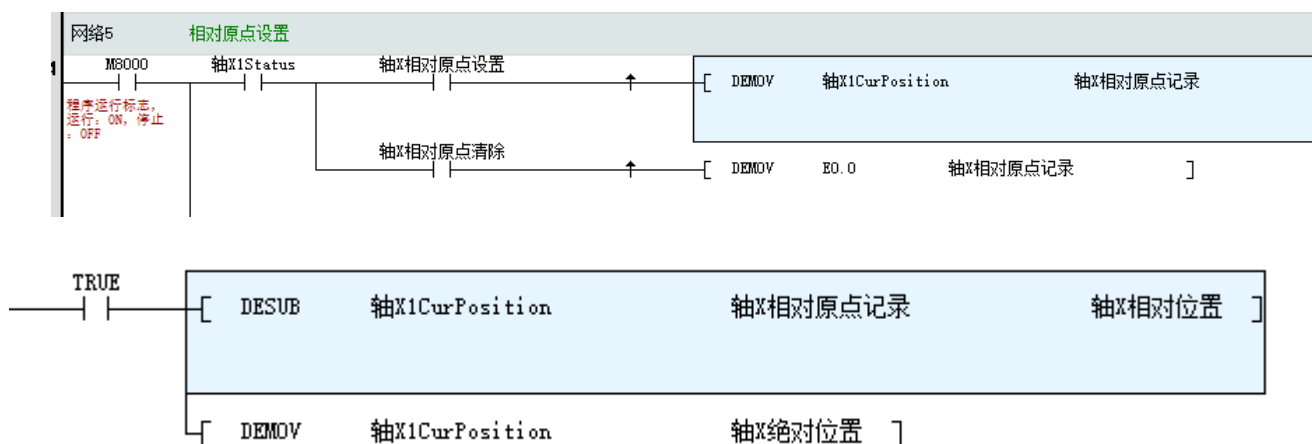
■ 轴X当前相对位置

```
1 轴X当前相对位置:=轴X相对位置;
```



■ 轴X当前相对零点

```
1 轴X当前相对零点:=轴X相对原点记录;
```



■ 轴X联动状态

```
1  // 联动开启
2  IF 轴X1CurSpeed = 0.0 AND 轴X2CurSpeed = 0.0 AND 轴X1Status AND 轴X2Status THEN
3      IF 轴X联动使能 AND ABS(轴X1CurPosition + 轴X2CurPosition) < 0.1 THEN
4          X轴关联触发 := 轴X联动使能;
5      END_IF;
6  END_IF;
7  // 联动关闭
8  IF NOT 轴X联动使能 THEN
9      X轴关联触发 := 轴X联动使能;
10 END_IF;
11
12 X轴解除关联触发 := NOT 轴X联动使能;
13
14 轴X联动状态 := X轴关联触发;
15
```

```
1 IF (轴X1CurPosition <= 轴X软件负限位 OR 轴X1CurPosition >= 轴X软件正限位) AND 轴X联动状态 THEN
2     轴X软件限位状态 := TRUE;
3 ELSE
4     轴X软件限位状态 := FALSE;
5 END_IF;
```

■ 轴X软件限位状态

```

1 // 轴X 绝对与相对定位接口 必须在X1 X2联动的情况下才能执行
2 轴X暂停沿触发(CLK := 轴X软件限位状态,Q => 轴X暂停触发结果);
3 IF 轴X软件限位状态 THEN
4     轴X1MoveDo := FALSE;
5     轴X1MoveXDDo := false;
6 END_IF;
7
8 if 齿轮比到达 AND
9     轴X联动使能 AND NOT 解除关联完成 AND 轴X1CurSpeed = 0.0 AND
10    NOT 轴X1MoveDo AND NOT 轴X1MoveXDDo and not 轴X1HomeDo and NOT 轴X1ZeroDo and
11    NOT 轴X1JogForward AND NOT 轴X1JogBackward AND NOT X轴急停触发 THEN
12
13    IF not 轴X软件限位状态 and
14        轴X绝对定位距离 <> 轴X1CurPosition AND 轴X绝对定位触发 AND NOT 轴X相对定位触发 then
15        轴X1MovePosition := 轴X绝对定位距离;
16        轴X1MoveDo := TRUE;
17    END_IF;
18    轴X相对定位沿触发(CLK := 轴X相对定位触发,Q => 轴X相对定位沿触发结果);
19    IF not 轴X软件限位状态 and
20        轴X相对定位距离 <> 0.0 AND not 轴X绝对定位触发 and 轴X相对定位沿触发结果 then
21        轴X1MoveXDPosition := 轴X相对定位距离;
22        轴X1MoveXDDo := TRUE;
23    END_IF;
24
25 END_IF;
26

```

```

1 // 轴X 点动 接口
2 IF 轴X软件限位状态 THEN
3     IF 轴X1CurPosition <= 轴X软件负限位 THEN 轴X1点动反转 := FALSE; END_IF;
4     IF 轴X1CurPosition >= 轴X软件正限位 THEN 轴X1点动正转 := FALSE; end_if ;
5 END_IF;
6
7 if ((齿轮比到达 AND
8     轴X联动使能 AND NOT 解除关联完成)OR NOT 轴X联动使能) AND
9     NOT 轴X1MoveDo AND NOT 轴X1MoveXDDo and not 轴X1HomeDo and NOT 轴X1ZeroDo AND 轴X1Status
10    AND NOT X轴超差时间到 AND NOT X轴急停触发 THEN
11    轴X1JogBackward := 轴X1点动反转;
12    轴X1JogForward := 轴X1点动正转;
13 END_IF;
14
15 IF NOT 齿轮比到达 AND NOT 轴X联动使能 AND NOT X轴超差时间到 AND
16    NOT 轴X2MoveDo AND NOT 轴X2MoveXDDo and not 轴X2HomeDo and NOT 轴X2ZeroDo AND 轴X2Status THEN
17    轴X2JogBackward := 轴X2点动反转;
18    轴X2JogForward := 轴X2点动正转;
19 END_IF;
20

```

```

1 // 联合定位的时候，两轴的位置超差就需要紧急停机
2 if 齿轮比到达 AND 轴X联动使能 AND NOT 解除关联完成 THEN
3     if ABS(轴X1CurPosition + 轴X2CurPosition) > 轴X超差阈值 THEN
4         X轴超差标志 := TRUE;
5     ELSE
6         X轴超差标志 := FALSE;

```

```

7      END_IF;
8  ELSE
9      X轴超差标志 := FALSE;
10     轴X1E_Stop := FALSE;
11 end_if;
12
13 TONR(IN := X轴超差标志,PT := 20,R := ,Q => X轴超差时间到,ET => );
14 if X轴超差时间到 THEN
15     X轴急停触发 := TRUE;
16 END_IF;
17 IF X轴急停触发 THEN
18     轴X绝对定位触发 := FALSE;
19     轴X相对定位触发 := FALSE;
20
21     轴X1点动反转 := FALSE ;
22     轴X1点动正转 := FALSE ;
23
24     轴X1MoveDo := FALSE;
25     轴X1MoveXDDo := FALSE;
26     轴X1HomeDo := FALSE;
27     轴X1ZeroDo := FALSE;
28 END_IF;
29

```

```

1  IF (轴X1CurPosition <= 轴X软件负限位 OR 轴X1CurPosition >= 轴X软件正限位) AND 轴X联动状态 THEN
2      轴X软件限位状态 := TRUE;
3  ELSE
4      轴X软件限位状态 := FALSE;
5  END_IF;

```